

SESSION 2

08.09.2024

Agenda

NumPy presentation

Online Quiz on NumPy

Panda presentation

Online Quiz on Panda

Tutorial Session

Data - relevant, irrelevant

Data analyst - clean

Clean - meaningful, then perform sorting , prediction process

Operands, operator

Data Clean relevant

Data	1	0	0
------	---	---	---

Analyst	0
---------	---

Social media

Scalar, vector - excel / csv

Int a[10] = {1,2,3,4};

NumPy

NumPy stands for **Numerical Python**.

NumPy is a Python library used for working with arrays.

It also has functions for working in domain of linear algebra, fourier transform, and matrices.

NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

NumPy is a Python library and is written partially in Python, but most of the parts that require fast computation are written in C or C++.

Tuple: (immutable)

A_tuple = (1,2,3)

List: (mutable)

A_list = [1,2,3]

Set: (unordered, unchangeable)

Set1 = {3,"Class",True}

Series: (1D array)

Example: One column in A table

Dictionary: {}

Humidity
32
33
34

Why NumPy

In Python we have lists that serve the purpose of arrays, but they are slow to process.

NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.

The array object in NumPy is called `ndarray`, it provides a lot of supporting functions that make working with `ndarray` very easy.

Arrays are very frequently used in data science, where speed and resources are very important.

NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.

This behavior is called locality of reference in computer science.

This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

Installation and Basics

```
pip install numpy
```

```
import numpy
```

```
arr = numpy.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

Output:

```
[1 2 3 4 5]
```

NumPy is usually imported under the **np** alias.

alias: In Python alias are an alternate name for referring to the same thing.

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

Output:

```
[1 2 3 4 5]
```


Find the NumPy version:

```
print(np.__version__)
```

Output:

1.26.4

Creating NumPy Arrays:

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
print(type(arr))
```

Output:

```
<class 'numpy.ndarray'>
```

To create an `ndarray`, we can pass a list, tuple or any array-like object into the `array()` method, and it will be converted into an `ndarray`:

```
#Creating NumPy array using tuple
```

```
arr = np.array((1, 2, 3, 4, 5))
```

```
print(arr)
```

Output:

```
[1 2 3 4 5]
```

Dimensions in Arrays

A dimension in arrays is one level of array depth (nested arrays).

Nested array are arrays that have arrays as their elements.

0-D Arrays

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

```
arr = np.array(42)
print(arr)
```

Output: 42

1-D Arrays

An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array. These are the most common and basic arrays.

Ex. Create a 1-D array containing the values 1,2,3,4,5:

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

2-D Arrays

An array that has 1-D arrays as its elements is called a 2-D array.

These are often used to represent matrix or 2nd order tensors.-> is a matrix that has n by n elements in a n-dimensional space.

NumPy has a whole sub module dedicated towards matrix operations called `numpy.mat`

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr)
```

Output:

```
[[1 2 3]
```

```
 [4 5 6]]
```

3-D arrays

An array that has 2-D arrays (matrices) as its elements is called 3-D array.

```
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])  
  
print(arr)
```

Output:

```
[[[1 2 3]
```

```
 [4 5 6]]
```

```
[[[1 2 3]
```

```
 [4 5 6]]]
```

Higher Dimensional Arrays

An array can have any number of dimensions.

When the array is created, you can define the number of dimensions by using the `ndmin` argument.

Example

Create an array with 5 dimensions and verify that it has 5 dimensions:

```
arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)

print('number of dimensions :', arr.ndim)
```

Output:

```
[[[[[1 2 3 4]]]]]

number of dimensions : 5
```

Exercise: Find the dimension??

```
import numpy as np
```

```
a = np.array(42)
```

```
b = np.array([1, 2, 3, 4, 5])
```

```
c = np.array([[1, 2, 3], [4, 5, 6]])
```

```
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
```

```
print(a.ndim)
```

```
print(b.ndim)
```

```
print(c.ndim)
```

```
print(d.ndim)
```

NumPy Functions

`arange()`: The code creates a NumPy array with range of elements

`eye()`: Creates an identity matrix in NumPy.

`ndim`: Prints the number of dimensions

`max()`, `maximum()`, and `amax()` : To find the maximum element in a NumPy array

`stack()`: The code stacks two NumPy arrays vertically

`reshape()` and `resize()`: To reshape a NumPy array.

`dot()`: Performs matrix multiplication between two NumPy arrays.

`argmax()` and `argmin()`: To find the indices of the maximum and minimum elements in a NumPy array.

`sum()`: To find the sum of the elements in a NumPy array.

`median()`: To find the median of a NumPy array.

`concatenate()`: Creates new NumPy array with the elements from both arrays concatenated.

`outer()`: product of two vectors a and b is a matrix where each element is the product of one element from a and one element from b

`dot()`: To compute the dot product of two NumPy arrays. The dot product of two vectors a and b is the sum of the products of the corresponding elements of a and b .

`inv()`: To find the inverse of a matrix in NumPy.

Panda in python

Pandas is a Python library used for working with data sets.

It has functions for cleaning,manipulating, analyzing, exploring, data.

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Why Pandas?

Pandas allows us to analyze big data and make conclusions based on statistical theories.

Pandas can clean messy data sets, and make them readable and relevant.

Relevant data is very important in data science.

Pandas gives answers about the data. Like:

- Is there a correlation between two or more columns?
- What is average value?
- Max value?
- Min value?

Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called *cleaning* the data.

Installation of Pandas

If you have [Python](#) and [PIP](#) already installed on a system, then installation of Pandas is very easy.

Install it using this command:

```
C:\Users\Your Name>pip install pandas
```

If this command fails, then use a python distribution that already has Pandas installed like, Anaconda, Spyder etc.

Once Pandas is installed, import it in your applications by adding the `import` keyword:

```
import pandas
```

Now Pandas is imported and ready to use.

Example:

```
import pandas
```

```
mydataset = {  
    'cars': ["BMW", "Volvo", "Ford"],  
    'passings': [3, 7, 2]  
}
```

```
myvar = pandas.DataFrame(mydataset)
```

```
print(myvar)
```

Pandas is usually imported under the `pd` alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the `as` keyword while importing:

```
import pandas as pd
```

```
import pandas as pd
```

```
mydataset = {
```

```
    'cars': ["BMW", "Volvo", "Ford"],
```

```
    'passings': [3, 7, 2]}
```

```
myvar = pd.DataFrame(mydataset)
```

```
print(myvar)
```

To print version:

```
print(pd.__version__)
```

Pandas Series

A Pandas Series is like a column in a table.

It is a one-dimensional array holding data of any type.

Ex: Creating simple pandas

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a)
```

```
print(myvar)
```

Labels

If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

Example

Return the first value of the Series:

```
print(myvar[0])
```


Create Labels

With the `index` argument, you can name your own labels.

Example

Create your own labels:

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a, index = ["x", "y", "z"])
```

```
print(myvar)
```

When you have created labels, you can access an item by referring to the label.

Example

Return the value of "y":

```
print(myvar["y"])
```

Key/Value Objects as Series

You can also use a key/value object, like a dictionary, when creating a Series.

Example

Create a simple Pandas Series from a dictionary:

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)
```

To select only some of the items in the dictionary, use the `index` argument and specify only the items you want to include in the Series.

Example

Create a Series using only data from "day1" and "day2":

```
calories = {"day1": 420, "day2": 380, "day3": 390}
```

```
myvar = pd.Series(calories, index = ["day1", "day2"])
```

```
print(myvar)
```

DataFrames

Data sets in Pandas are usually multi-dimensional tables, called DataFrames.

Series is like a column, a DataFrame is the whole table.

Example

Create a DataFrame from two Series:

```
import pandas as pd

data = {

    "calories": [420, 380, 390],

    "duration": [50, 40, 45]

}

myvar = pd.DataFrame(data)

print(myvar)
```

A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.

Create a simple Pandas DataFrame:

```
import pandas as pd

data = {

    "calories": [420, 380, 390],

    "duration": [50, 40, 45]

}

#load data into a DataFrame object:

df = pd.DataFrame(data)

print(df)
```

Locate Row

DataFrame is like a table with rows and columns.

Pandas use the `loc` attribute to return one or more specified row(s)

Example

Return row 0: `#refer to the row index:`

```
print(df.loc[0])
```

Output:

```
calories    420
```

```
duration     50
```

```
Name: 0, dtype: int64
```

```
#use a list of indexes:
```

```
print(df.loc[[0, 1]])
```

Output:

	calories	duration
0	420	50
1	380	40

Named Indexes

With the `index` argument, you can name your own indexes.

```
data = {  
    "calories": [420, 380, 390],  
    "duration": [50, 40, 45]  
}  
  
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])  
  
print(df)
```

Output:

	calories	duration
day1	420	50
day2	380	40
day3	390	45

Locate Named Indexes

Use the named index in the `loc` attribute to return the specified row(s).

```
print(df.loc["day2"])
```

Output:

```
calories    380
```

```
duration     40
```

```
Name: day2, dtype: int64
```

Load Files Into a DataFrame

Load a comma separated file (CSV file) into a DataFrame:

```
df = pd.read_csv('data.csv')
```

```
print(df)
```

Read CSV Files

A simple way to store big data sets is to use CSV files (comma separated files).

CSV files contains plain text and is a well know format that can be read by everyone including Pandas.

In our examples we will be using a CSV file called 'data.csv'.

Load the CSV into a DataFrame:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df.to_string())
```

use `to_string()` to print the entire DataFrame.

If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows:

max_rows

The number of rows returned is defined in Pandas option settings.

You can check your system's maximum rows with the `pd.options.display.max_rows` statement.

Example

Check the number of maximum returned rows:

```
import pandas as pd
```

```
print(pd.options.display.max_rows)
```

In my system the number is 60, which means that if the DataFrame contains more than 60 rows, the `print(df)` statement will return only the headers and the first and last 5 rows.

You can change the maximum rows number with the same statement.

Example

Increase the maximum number of rows to display the entire DataFrame:

```
import pandas as pd
```

```
pd.options.display.max_rows = 9999
```

```
df = pd.read_csv('data.csv')
```

```
print(df)
```

Pandas - Analyzing DataFrames

Viewing the Data

One of the most used method for getting a quick overview of the DataFrame, is the `head()` method.

The `head()` method returns the headers and a specified number of rows, starting from the top.

Example

Printing the first 10 rows of the DataFrame:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df.head(10))
```


Download [data.csv](#), or open [data.csv](#) in your browser.

Lab 3:

Use Pandas to load a CSV file, display the first 10 rows, and perform basic data manipulation (sorting, filtering).